

to-understand source code.

In a blend tree, a different set of problems arise. Thanks to the tree structure, the clips are grouped naturally into functional units. Custom tree nodes can encapsulate complex character motions. These are both helpful advantages over the flat weighted average approach. However, the control parameters are buried within the tree. Code that wishes to control the horizontal look-at direction of the head and eyes needs a priori knowledge of the structure of the blend tree so that it can find the appropriate nodes in the tree in order to control their parameters.

Different animation engines solve these problems in different ways. Here are some examples:

- *Node search.* Some engines provide a way for higher-level code to *find* blend nodes in the tree. For example, relevant nodes in the tree can be given special names, such as “HorizAim” for the node that controls horizontal weapon aiming. The control code can simply search the tree for a node of a particular name; if one is found, then we know what effect adjusting its blend weight will have.
- *Named variables.* Some engines allow names to be assigned to the individual control parameters. The controlling code can look up a control parameter by name in order to adjust its value.
- *Control structure.* In other engines, a simple data structure, such as an array of floating-point values or a C struct, contains all of the control parameters for the entire character. The nodes in the blend tree(s) are connected to particular control parameters, either by being hard-coded to use certain struct members or by looking up the parameters by name or index.

Of course, there are many other alternatives as well. Every animation engine tackles this problem in a slightly different way, but the net effect is always roughly the same.

12.11 Constraints

We’ve seen how action state machines can be used to specify complex blend trees and how a transition matrix can be used to control how transitions between states should work. Another important aspect of character animation control is to constrain the movement of the characters and/or objects in the scene in various ways. For example, we might want to constrain a weapon so that it always appears to be in the hand of the character who is carrying it.

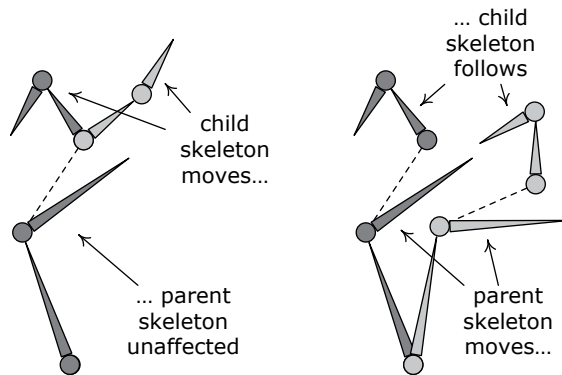


Figure 12.59. An attachment, showing how movement of the parent automatically produces movement of the child but not vice versa.

We might wish to constrain two characters so that they line up properly when shaking hands. A character's feet are often constrained so that they line up with the floor, and its hands might be constrained to line up with the rungs on a ladder or the steering wheel of a vehicle. In this section, we'll take a brief look at how these constraints are handled in a typical animation system.

12.11.1 Attachments

Virtually all modern game engines permit objects to be attached to one another. At its simplest, object-to-object attachment involves constraining the position and/or orientation of a particular joint J_A within the skeleton of object A so that it coincides with a joint J_B in the skeleton of object B. An attachment is usually a parent-child relationship. When the parent's skeleton moves, the child object is adjusted to satisfy the constraint. However, when the child moves, the parent's skeleton is usually not affected. This is illustrated in Figure 12.59.

Sometimes it can be convenient to introduce an *offset* between the parent joint and the child joint. For example, when placing a gun into a character's hand, we could constrain the "Grip" joint of the gun so that it coincides with the "RightWrist" joint of the character. However, this might not produce the correct alignment of the gun with the hand. One solution to this problem is to introduce a special joint into one of the two skeletons. For example, we could add a "RightGun" joint to the character's skeleton, make it a child of the "RightWrist" joint, and position it so that when the "Grip" joint of the gun is constrained to it, the gun looks like it is being held naturally by the character. The problem with this approach, however, is that it increases the

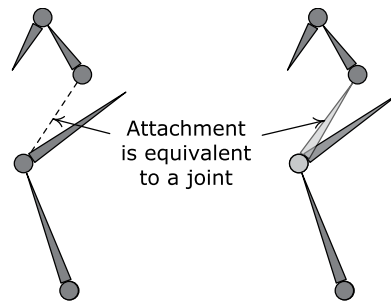


Figure 12.60. An attach point acts like an extra joint between the parent and the child.

number of joints in the skeleton. Each joint has a processing cost associated with animation blending and matrix palette calculation and a memory cost for storing its animation keys. So adding new joints is often not a viable option.

We know that an additional joint added for attachment purposes will not contribute to the pose of the character—it merely introduces an additional transform between the parent and child joint in an attachment. What we really want, then, is a way to mark certain joints so that they can be ignored by the animation blending pipeline but can still be used for attachment purposes. Such special joints are sometimes called *attach points*. They are illustrated in Figure 12.60.

Attach points might be modeled in Maya just like regular joints or locators, although many game engines define attach points in a more convenient manner. For example, they might be specified as part of the action state machine text file or via a custom GUI within the animation authoring tool. This allows the animators to focus only on the joints that affect the look of the character, while the power to control attachments is put conveniently into the hands of the people who need it—the game designers and the engineers.

12.11.2 Interobject Registration

The interactions between game characters and their environments is growing ever more complex and nuanced with each new title. Hence, it is important to have a system that allows characters and objects to be aligned with one another when animating. Such a system can be used for in-game cinematics and interactive gameplay elements alike.

Imagine that an animator, working in Maya or some other animation tool, sets up a scene involving two characters and a door object. The two characters shake hands, and then one of them opens the door and they both walk through it. The animator can ensure that all three actors in the scene line up perfectly.

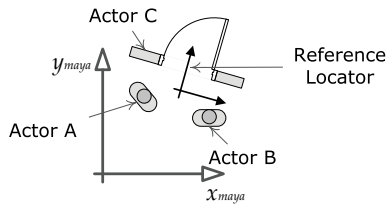


Figure 12.61. Original Maya scene containing three actors and a reference locator.

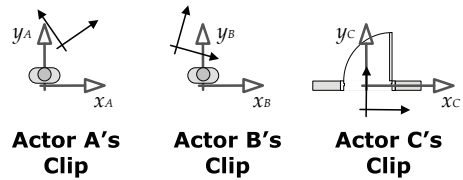


Figure 12.62. The reference locator is encoded in each actor's animation file.

However, when the animations are exported, they become three separate clips, to be played on three separate objects in the game world. The two characters might have been under AI or player control prior to the start of this animated sequence. How, then, can we ensure that the three objects line up correctly with one another when the three clips are played back in-game?

12.11.2.1 Reference Locators

One good solution is to introduce a common reference point into all three animation clips. In Maya, the animator can drop a *locator* (which is just a 3D transform, much like a skeletal joint) into the scene, placing it anywhere that seems convenient. Its location and orientation are actually irrelevant, as we'll see. The locator is tagged in some way to tell the animation export tools that it is to be treated specially.

When the three animation clips are exported, the tools store the position and orientation of the reference locator, expressed in coordinates that are relative to the local object space of each actor, into all three clip's data files. Later, when the three clips are played back in-game, the animation engine can look up the relative position and orientation of the reference locator in all three clips. It can then transform the origins of the three objects in such a way as to make all three reference locators coincide in world space. The reference locator acts much like an *attach point* (Section 12.11.1) and, in fact, could be implemented as one. The net effect—all three actors now line up with one another, exactly as they had been aligned in the original Maya scene.

Figure 12.61 illustrates how the door and the two characters from the above example might be set up in a Maya scene. As shown in Figure 12.62, the reference locator appears in each exported animation clip (expressed in that actor's local space). In-game, these local-space reference locators are aligned to a fixed world-space locator in order to realign the actors, as shown in Figure 12.63.

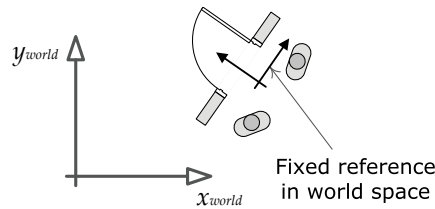


Figure 12.63. At runtime, the local-space reference transforms are aligned to a world-space reference locator, causing the actors to line up properly.

12.11.2.2 Finding the World-Space Reference Location

We've glossed over one important detail here—who decides what the world-space position and orientation of the reference locator should be? Each animation clip provides the reference locator's transform in the coordinate space of *its actor*. But we need some way to define where that reference locator should be in world space.

In our example with the door and the two characters shaking hands, one of the actors is fixed in the world (the door). So one viable solution is to ask the door for the location of the reference locator and then align the two characters to it. The commands to accomplish this might look similar to the following pseudocode.

```
void playShakingHandsDoorSequence(
    Actor& door,
    Actor& characterA,
    Actor& characterB)
{
    // Find the world-space transform of the reference
    // locator as specified in the door's animation.
    Transform refLoc = getReferenceLocatorWs(door,
        "shake-hands-door");

    // Play the door's animation in-place. (It's
    // already in the correct place.)
    playAnimation("shake-hands-door", door);

    // Play the two characters' animations relative to
    // the world-space reference locator obtained from
    // the door.
    playAnimationRelativeToReference(
        "shake-hands-character-a", characterA, refLoc);
    playAnimationRelativeToReference(
        "shake-hands-character-b", characterB, refLoc);
}
```

Another option is to define the world-space transform of the reference locator independently of the three actors in the scene. We could place the reference locator into the world using our world-building tool, for example (see Section 15.3). In this case, the pseudocode above should be changed to look something like this:

```
void playShakingHandsDoorSequence(
    Actor& door,
    Actor& characterA,
    Actor& characterB,
    Actor& refLocatorActor)
{
    // Find the world-space transform of the reference
    // locator by simply querying the transform of an
    // independent actor (presumably placed into the
    // world manually).
    Transform refLoc = getActorTransformWs(
        refLocatorActor);

    // Play all animations relative to the world-space
    // reference locator obtained above.
    playAnimationRelativeToReference("shake-hands-door",
        door, refLoc);
    playAnimationRelativeToReference(
        "shake-hands-character-a", characterA, refLoc);
    playAnimationRelativeToReference(
        "shake-hands-character-b", characterB, refLoc);
}
```

12.11.3 Grabbing and Hand IK

Even after using an attachment to connect two objects, we sometimes find that the alignment does not look exactly right in-game. For example, a character might be holding a rifle in her right hand, with her left hand supporting the stock. As the character aims the weapon in various directions, we may notice that the left hand no longer aligns properly with the stock at certain aim angles. This kind of joint misalignment is caused by LERP blending. Even if the joints in question are aligned perfectly in clip A and in clip B, LERP blending does not guarantee that those joints will be in alignment when A and B are blended together.

One solution to this problem is to use *inverse kinematics* (IK) to correct the position of the left hand. The basic approach is to determine the desired target position for the joint in question. IK is then applied to a short chain of joints (usually two, three or four joints), starting with the joint in question and

progressing up the hierarchy to its parent, grandparent and so on. The joint whose position we are trying to correct is known as the *end effector*. The IK solver adjusts the orientations of the end effector's parent joint(s) in order to get the end effector as close as possible to the target.

The API for an IK system usually takes the form of a request to enable or disable IK on a particular chain of joints, plus a specification of the desired target point. The actual IK calculation is usually done internally by the low-level animation pipeline. This allows it to do the calculation at the proper time—namely, after intermediate local and global skeletal poses have been calculated but before the final matrix palette calculation.

Some animation engines allow IK chains to be defined a priori. For example, we might define one IK chain for the left arm, one for the right arm and two for the two legs. Let's assume for the purposes of this example that a particular IK chain is identified by the name of its end-effector joint. (Other engines might use an index or handle or some other unique identifier, but the concept remains the same.) The function to enable an IK calculation might look something like this:

```
void enableIkChain(Actor& actor,
                  const char* endEffectorJointName,
                  const Vector3& targetLocationWs);
```

and the function to disable an IK chain might look like this:

```
void disableIkChain(Actor& actor,
                   const char* endEffectorJointName);
```

IK is usually enabled and disabled relatively infrequently, but the world-space target location must be kept up-to-date every frame (if the target is moving). Therefore, the low-level animation pipeline always provides some mechanism for updating an active IK target point. For example, the pipeline might allow us to call `enableIkChain()` multiple times. The first time it is called, the IK chain is enabled, and its target point is set. All subsequent calls simply update the target point. Another way to keep IK targets up-to-date is to link them to dynamic objects in the game. For example, an IK target might be specified as a handle to a rigid game object, or a joint within an animated object.

IK is well-suited to making minor corrections to joint alignment when the joint is already reasonably close to its target. It does not work nearly as well when the error between a joint's desired location and its actual location is large. Note also that most IK algorithms solve only for the *position* of a joint. You may

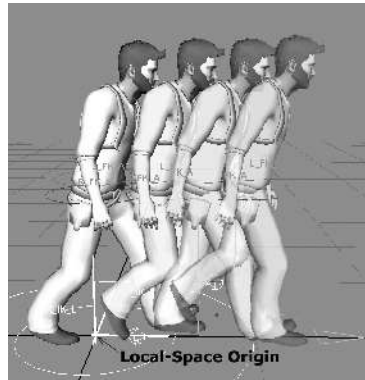


Figure 12.64. In the animation authoring package, the character moves forward in space, and its feet appear grounded. Image courtesy of Naughty Dog, Inc. (UNCHARTED: Drake's Fortune © 2007/© SIE. Created and developed by Naughty Dog.)

need to write additional code to ensure that the *orientation* of the end effector aligns properly with its target as well. IK is not a cure-all, and it may have significant performance costs. So always use it judiciously.

12.11.4 Motion Extraction and Foot IK

In games, we usually want the locomotion animations of our characters to look realistic and “grounded.” One of the biggest factors contributing to the realism of a locomotion animation is whether or not the feet slide around on the ground. *Foot sliding* can be overcome in a number of ways, the most common of which are *motion extraction* and *foot IK*.

12.11.4.1 Motion Extraction

Let’s imagine how we’d animate a character walking forward in a straight line. In Maya (or his or her animation package of choice), the animator makes the character take one complete step forward, first with the left foot and then with the right foot. The resulting animation clip is known as a *locomotion cycle*, because it is intended to be looped indefinitely, for as long as the character is walking forward in-game. The animator takes care to ensure that the feet of the character appear grounded and don’t slide as it moves. The character moves from its initial location on frame 0 to a new location at the end of the cycle. This is shown in Figure 12.64.

Notice that the local-space origin of the character remains fixed during the entire walk cycle. In effect, the character is “leaving his origin behind him” as he takes his step forward. Now imagine playing this animation as a loop. We



Figure 12.65. Walk cycle after zeroing out the root joint's forward motion. Image courtesy of Naughty Dog, Inc. (UNCHARTED: Drake's Fortune © 2007/© SIE. Created and developed by Naughty Dog.)

would see the character take one complete step forward, and then pop back to where he was on the first frame of the animation. Clearly this won't work in-game.

To make this work, we need to remove the forward motion of the character, so that his local-space origin remains roughly under the center of mass of the character at all times. We could do this by zeroing out the forward translation of the root joint of the character's skeleton. The resulting animation clip would make the character look like he's "moonwalking," as shown in Figure 12.65.

In order to get the feet to appear to "stick" to the ground the way they did in the original Maya scene, we need the character to move forward by just the right amount each frame. We could look at the distance the character moved, divide by the amount of time it took for him to get there, and hence find his average movement speed. But a character's forward speed is not constant when walking. This is especially evident when a character is limping (quick forward motion on the injured leg, followed by slower motion on the "good" leg), but it is true for all natural-looking walk cycles.

Therefore, before we zero out the forward motion of the root joint, we first save the animation data in a special "extracted motion" channel. This data can be used in-game to move the local-space origin of the character forward by the exact amount that the root joint had moved in Maya each frame. The net result is that the character will walk forward exactly as he was authored, but now his local-space origin comes along for the ride, allowing the animation to loop properly. This is shown in Figure 12.66.

If the character moves forward by 4 feet in the animation and the animation

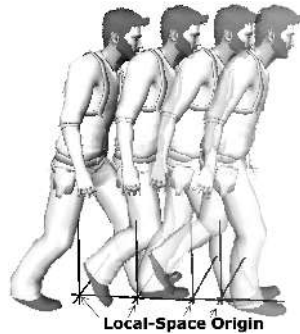


Figure 12.66. Walk cycle in-game, with extracted root motion data applied to the local-space origin of the character. Image courtesy of Naughty Dog, Inc. (UNCHARTED: Drake's Fortune © 2007/© SIE. Created and developed by Naughty Dog.)

takes one second to complete, then we know that the character is moving at an average speed of 4 feet/second. To make the character walk at a different speed, we can simply scale the playback rate of the walk cycle animation. For example, to make the character walk at 2 feet/second, we can simply play the animation at half speed ($R = 0.5$).

12.II.4.2 Foot IK

Motion extraction does a good job of making a character's feet appear grounded when it is moving in a straight line (or, more correctly, when it moves in a path that exactly matches the path animated by the animator). However, a real game character must be turned and moved in ways that don't coincide with the original hand-animated path of motion (e.g., when moving over uneven terrain). This results in additional foot sliding.

One solution to this problem is to use IK to correct for any sliding in the feet. The basic idea is to analyze the animations to determine during which periods of time each foot is fully in contact with the ground. At the moment a foot contacts the ground, we note its world-space location. For all subsequent frames while that foot remains on the ground, we use IK to adjust the pose of the leg so that the foot remains fixed to the proper location. This technique sounds easy enough, but getting it to look and feel right can be very challenging. It requires a lot of iteration and fine-tuning. And some natural human motions—like leading into a turn by increasing your stride—cannot be produced by IK alone.

In addition, there is a big trade-off between the *look* of the animations and the *feel* of the character, particularly for a human-controlled character. It's generally more important for the player character control system to feel responsive

and fun than it is for the character's animations to look perfect. The upshot is this: Do not take the task of adding foot IK or motion extraction to your game lightly. Budget time for a lot of trial and error, and be prepared to make trade-offs to ensure that your player character not only looks good but *feels* good as well.

12.11.5 Other Kinds of Constraints

There are plenty of other possible kinds of constraint systems that can be added to a game animation engine. Some examples include:

- *Look-at.* This is the ability for characters to look at points of interest in the environment. A character might look at a point with only his or her eyes, with eyes and head, or with eyes, head and a twist of the entire upper body. Look-at constraints are sometimes implemented using IK or procedural joint offsets, although a more natural look can often be achieved via additive blending.
- *Cover registration.* This is the ability for a character to align perfectly with an object that is serving as cover. This is often implemented via the reference locator technique described above.
- *Cover entry and departure.* If a character can take cover, animation blending and custom entry and departure animations must usually be used to get the character into and out of cover.
- *Traversal aids.* The ability for a character to navigate over, under, around or through obstacles in the environment can add a lot of life to a game. This is often done by providing custom animations and using a reference locator to ensure proper registration with the obstacle being overcome.